DANIYA ALI

SEC D

CT – 25159

MR MUDASSIR AHMED

DSA LAB 4

QUESTION NO 1

SOURCE CODE:

Greedy > palindrome_stack.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;
bool isPalindrome(string s){

    if(s.empty() || s.size() == 1) return true;

    stack<char>st;

    for (int i=0; i<s.size(); i++){
        if(isalnum(s[i])){
            st.push(tolower(s[i]));
        }
    }

    int i=0;
    while(!st.empty()){
        while(i<s.length() && !isalnum(s[i])) i++;
        if (i<s.length() && tolower(s[i]) != st.top()) return false;
        st.pop();
        i++;
    }

    return true;
}
int main(){
    string s = "A man, a plan, a canal: Panama";

    if(isPalindrome(s)) {
        cout << "It is a palindrome!" << endl;
    } else {
        cout << "It is NOT a palindrome." << endl;
    }
    return 0;
}
```

OUTPUT:

```
PS C:\Users\User\OneDrive\Desktop\DSA> cd "c:\Users\Us
_stack }
It is a palindrome!
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy>
```

QUESTION NO 2

SOURCE CODE:

Greedy > backspace_stack.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;
stack<char> buildStack(string str) {
    stack<char> st;
    for (char c : str) {
        if (c != '#') {
            st.push(c);
        } else if (!st.empty()) {
            st.pop();
        }
    }
    return st;
}

bool isEqual(string s, string t) {
    stack<char> s1 = buildStack(s);
    stack<char> s2 = buildStack(t);

    if (s1.size() != s2.size()) return false;

    while (!s1.empty()) {
        if (s1.top() != s2.top()) return false;
        s1.pop();
        s2.pop();
    }

    return true;
}

int main() {
    string s = "ab#c";
    string t = "ad#c";

    if (isEqual(s, t)) {
        cout << "The strings are equal!" << endl;
    } else {
        cout << "The strings are NOT equal" << endl;
    }
    return 0;
}
```

OUTPUT:

```
It is a palindrome!
The strings are equal!
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy>
```

QUESTION NO 3

SOURCE CODE:

Greedy > permutation.cpp

```cpp
#include <iostream>
#include <vector>
using namespace std;

void permuteHelper(vector<int>& nums, int index, vector<vector<int>>& res) {
    if (index == nums.size()) {
        res.push_back(nums);
        return;
    }
    for (int i = index; i < nums.size(); i++) {
        swap(nums[index], nums[i]);
        permuteHelper(nums, index + 1, res);
        swap(nums[index], nums[i]); // Backtrack
    }
}

vector<vector<int>> permute(vector<int>& nums) {
    vector<vector<int>> res;
    permuteHelper(nums, 0, res);
    return res;
}

int main() {
    vector<int> nums = {1, 2, 3};
    vector<vector<int>> res = permute(nums);
    for (const auto& vec : res) {
        cout << "[ ";
        for (int num : vec) cout << num << " ";
        cout << "] ";
    }
    cout << endl;
    return 0;
}
```

OUTPUT:

```
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy> cd "c:\Users\User\
[ 1 2 3 ] [ 1 3 2 ] [ 2 1 3 ] [ 2 3 1 ] [ 3 2 1 ] [ 3 1 2 ]
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy>
```

QUESTION NO 4

SOURCE CODE:

```cpp
#include <iostream>
#include <vector>
#include <string>
using namespace std;

bool dfs(vector<vector<char>>& board, string& word, int r, int c, int index) {
    if (index == word.length()) return true;
    if (r < 0 || c < 0 || r >= board.size() || c >= board[0].size() || board[r][c] != word[index])
        return false;

    char temp = board[r][c];
    board[r][c] = '#'; // Mark as visited

    bool found = dfs(board, word, r + 1, c, index + 1) ||
                 dfs(board, word, r - 1, c, index + 1) ||
                 dfs(board, word, r, c + 1, index + 1) ||
                 dfs(board, word, r, c - 1, index + 1);

    board[r][c] = temp; // Backtrack
    return found;
}

bool exist(vector<vector<char>>& board, string word) {
    int rows = board.size();
    int cols = board[0].size();
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (dfs(board, word, r, c, 0)) return true;
        }
    }
    return false;
}

int main() {
    vector<vector<char>> board = {
        {'A', 'B', 'C', 'E'},
        {'S', 'F', 'C', 'S'},
        {'A', 'D', 'E', 'E'}
    };
    string word = "ABCCED";
    if (exist(board, word)) {
        cout << "Output: true" << endl;
    } else {
        cout << "Output: false" << endl;
    }
```

OUTPUT:

```
Output: true
```